

PrimeWave™ Design Environment Simulator Integration Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	5
Related Products, Publications, and Trademarks	5
Conventions	6
Customer Support	7
Statement on Inclusivity and Diversity	8

1. Overview	9
Initializing the PrimeWave Design Environment	10
Initial Setup	10
Late Integration Files and Directory Structure Setup	11
Defining Integration Files and Directory Structure	11

2. Creating a New Integration	13
Defining the Simulator	14
Setting Up the startup.tcl File	15
Setting Preferences	16
Creating Menus	18
Defining Shortcut Keys	18
Customizing Netlists	19
Setting Netlists for Integrations	19
Defining Simulator Analyses	20
Creating Dialog Boxes and Registering Analysis Fields	23
Creating Preferences for Analysis Fields	25
Defining Selection Functions for Analysis Fields	25
Validating Analysis Fields	27
Defining Dependencies between Analyses	28
Defining Synopsis Procedures for Analysis	29
Defining Displayable Fields	29
Customizing Dialog Boxes	31
Creating New Dialog Boxes	32

Contents

Specifying Simulator Options	32
Creating Simulator Input Files	34
Defining Simulator Run Commands	35
Defining Post-Processed Simulation Results	36
Mapping Data	36
Defining Results Maps for Simulator Output Files	38
Mapping Operating Point Parameter Name	39
Customizing the Results Analyzer	40
Customizing Existing Integrations	43
A. Integration Objects and Callbacks	44
Objects	44
saSimulator	45
saAnalysisDef	46
saAnalysisInfo	46
saResultsMap	47
saResultsInfo	47
saResults	48
saResultFile	48
saSelectionDef	49
saSelectionGroup	49
saSignalObject	50
saSelection	50
saExpressionDefProcArgs	51
saExpressionDef	52
Callbacks	53
Simulator Callbacks	54
Analysis Callbacks	55
Environment Option Callbacks	56
Save Option Callbacks	56
Data Map Callbacks	57
Error Checking Procedures	57
Results Analyzer Registration Functions	58
Options XML Schema Definition File	60
Using Markup Syntax	61

About the PrimeWave Design Environment Simulator Integration Guide

The *PrimeWave Design Environment Simulator Integration Guide* describes the PrimeWave Design Environment interface for integrating new simulators. You can also customize the provided integrations to suit your flow. This document provides instructions and examples of how to use the integration interface.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeWave Design Environment Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeWave™ Design Environment tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® PrimeWave™ Design Environment

Synopsys® Custom Compiler™

PrimeSim™ Continuum™

PrimeSim™

PrimeSim™ Pro™

- PrimeSim™ SPICE™
- PrimeSim™ HSPICE®
- PrimeSim™ XA™
- PrimeSim™ CCK™
- PrimeSim™ MOSRA™
- VCS® PrimeSim™ AMS™
- Synopsys® FineSim®
- Synopsys® HSPICE®
- Synopsys® VCS®
- Synopsys® Verdi®
- Cadence® Spectre® Circuit Simulator
- MathWorks® MATLAB

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .

Convention	Description
	Indicates a choice among alternatives, such as low medium high
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Overview

The PrimeWave Design Environment provides an interface for users to customize integrations or create new integrations.

The integration interface creates new integrations using PrimeWave Design Environment and native Tcl commands.

Integrations can be customized in the following areas:

- Creating and setting up the testbenches for simulation.
- Creating a Simulator Input File using the following components:
 - Structural netlist of the design.
 - User-defined functions such as analysis statements and save options that are required for the simulator.
 - User-defined functions such as probed signals and design variables.
- Running the netlister and simulator in local or distributed mode.
- Running advanced analyses such as parametric, corners, and Monte Carlo.
- Post processing the simulation results.
- Providing a command-line interface for all these capabilities.

By default, you can access several simulator integrations that are included with the PrimeWave Design Environment installation by choosing **Setup > Simulator** from the PrimeWave Design Environment main menu bar.

The Tcl files defining the integrations are located in the following directory:

```
$SYNOPSYS_CUSTOM_INSTALL/primewave
```

Each integration is defined in a directory named after the integration name. For example, all the files defining the integration for PrimeSim can be found in the \$SYNOPSYS_CUSTOM_INSTALL/primewave/PrimeSim directory.

This section covers the following topics:

- [Initializing the PrimeWave Design Environment](#)
- [Defining Integration Files and Directory Structure](#)

Initializing the PrimeWave Design Environment

Menus, preferences, key bindings, and simulator integrations that are located in the installation or custom directories must first be initialized into the PrimeWave Design Environment.

The PrimeWave Design Environment initialization consists of two parts:

- [Initial Setup](#)
- [Late Integration Files and Directory Structure Setup](#)

Initial Setup

At initial startup, the system loads preferences and bindings using the standard Custom Compiler INSTALL, SITE, PROJECT, USER, and LOCAL levels load sequences.

For more information on the Custom Compiler load sequence, refer to Starting Sessions in the *Custom Compiler Environment User Guide*.

All viewer and simulator integrations are located and registered at startup. The netlisters are initialized via the netlist-based Tool Integration interface. The `startup.tcl` file, which is located in the `$SYNOPSIS_CUSTOM_INSTALL/sae` directory, performs the following actions as the PrimeWave Design Environment first opens:

- Declares the generic PrimeWave Design Environment preferences by sourcing the `$SYNOPSIS_CUSTOM_INSTALL /sae/preferences.tcl` file.
- Registers the generic PrimeWave Design Environment menus and tool bars by sourcing the `$SYNOPSIS_CUSTOM_INSTALL /sae/menus.tcl` file.
- Registers the generic PrimeWave Design Environment key bindings by sourcing the `$SYNOPSIS_CUSTOM_INSTALL /sae/bindings.tcl` file.
- Finds all the simulators integrated into the PrimeWave Design Environment. Installed and custom integrations located at the SITE, PROJECT, USER, and LOCAL levels.
- Sources the `startup.tcl` from the simulation integration directory for each simulator integrated into the PrimeWave Design Environment.
- Finds and registers all the viewer integrations.

Late Integration Files and Directory Structure Setup

The late startup occurs when you invoke the PrimeWave Design Environment or when you choose an integration from the **Select Simulator** dialog box. The selected simulator and viewer integrations are then initialized. This loads secondary netlisters and simulator-specific menus, shortcut keys, and preferences.

Defining Integration Files and Directory Structure

When any integration is defined, whether in the installation or a custom integration in the SITE, PROJECT or USER directories, you must follow a prescribed directory structure so that those integrations are located, registered, and initialized correctly. Several required files (and some optional files) must be present, because they are used to define the features of the integration.

To define an integration directory structure, create a directory named "primewave" in the SITE, PROJECT, USER or LOCAL directories to contain new or customized integrations.

For example:

```
$SYNOPSYS_CUSTOM_SITE/primewave
```

Each custom integration must reside in a directory named after the integration name. For example, the following directories are created to add new integrations called SPICE3 and ngSpice:

```
$SYNOPSYS_CUSTOM_SITE/primewave/SPICE3  
$SYNOPSYS_CUSTOM_SITE/primewave/ngSpice
```

An integrations.tcl file must be defined in the primewave directory to identify any new integrations you define:

```
$SYNOPSYS_CUSTOM_SITE/primewave/integrations.tcl  
db::init primewave/SPICE3/startup.tcl  
db::init primewave/ngSpice/startup.tcl
```

You can create several files to define the features of an integration in a simulator integration directory. The following table lists the files and describes the function of each file.

File Name	Description	Optional
startup.tcl	Sources the preferences.tcl file, which is used for declaring the simulator-specific preferences. Also sources the other Tcl files located in the simulator integration directory.	No

File Name	Description	Optional
bindings.tcl	Modifies the standard PrimeWave Design Environment shortcut keys. This file is called when the PrimeWave Design Environment first opens as well as every time the simulator is changes.	Yes
menus.tcl	Modifies the PrimeWave Design Environment banner menus and tool bars. This file is called when the PrimeWave Design Environment first opens, as well as every time the simulator is changed.	Yes
preferences.tcl	Creates simulator integrator specific preferences via commands. This file is loaded as part of the PrimeWave Design Environment startup. Use <code>sa::makePrefName</code> to automatically generate the name for the preferences. If the generated preference name is not suitable for certain fields, the preference name must be specified as part of the field registration.	Yes
environment_options.tcl	Contains Tcl procedures for customizing (adding additional fields) the environment options dialog box for the target simulator needs.	Yes
<analysis>.tcl	Contains Tcl routines and variables for declaring and managing simulator analyses.	Yes
save_options.tcl	Contains Tcl routines and variables which define the save options for the simulator.	Yes
simulator.tcl	Contains Tcl routines that contain the simulator registration. Also registers the type of external files in the Include Files dialog box that are supported (among other types).	No
<simName>.tcl	Contains Tcl routines and variables, which defines the integration for a simulation tool. This file is required because it implements the run command, which is used to run simulations.	No
user.tcl	Contains additional routines and variables, which define new custom dialog boxes.	Yes
< simName>.template	Configures the structure of the final simulator input file (final netlist).	No
language.tcl	Contains procedures for writing corresponding netlist lines for PrimeWave Design Environment settings.	Yes
pa_expressions.tcl	Registers analysis expressions for the Results Analyzer Plot Assistant.	Yes

Note:

You can create other files not included in this list and include them in the startup.tcl file.

2

Creating a New Integration

New integrations can be created at the SITE, PROJECT, USER, and LOCAL levels. These are initialized as described in [Initializing the PrimeWave Design Environment](#).

This document includes a sample simulator integration called SPICE3.

Defining a new integration can involve the following tasks:

- [Defining the Simulator](#)
- [Setting Up the startup.tcl File](#)
- [Setting Preferences](#)
- [Creating Menus](#)
- [Defining Shortcut Keys](#)
- [Customizing Netlisters](#)
- [Setting Netlisters for Integrations](#)
- [Defining Simulator Analyses](#)
- [Customizing Dialog Boxes](#)
- [Creating New Dialog Boxes](#)
- [Specifying Simulator Options](#)
- [Creating Simulator Input Files](#)
- [Defining Simulator Run Commands](#)
- [Defining Post-Processed Simulation Results](#)
- [Customizing the Results Analyzer](#)
- [Customizing Existing Integrations](#)

Defining the Simulator

For each integration, you need to define and configure an `saSimulator` object. You must configure the `saSimulator` object with the following information before it can be used:

- final netlist
- netlist template
- log file
- list of analysis types the simulator supports

The object name must be the integration name, which is also the same as the directory name where the integration code is stored.

You must call `sa::createSimulator` to create an instance of the `saSimulator` object for the simulator. Refer to the [Integration Objects and Callbacks](#) for a complete description of the attributes of the `saSimulator` object and the arguments of the `sa::createSimulator` command.

You can use the `simulator.tcl` file to create and configure the `saSimulator` object. The following example uses the `sa::createSimulator` command and configures the `saSimulator` object:

```
# Copyright (c) 2004-2015 Synopsys, Inc. This Custom Compiler
# software and the associated documentation are
# confidential and proprietary to Synopsys, Inc. Your use or
# disclosure of this Custom Compiler software is subject
# to the terms and conditions of a written license agreement
# between you, or your company, and Synopsys, Inc.
# Project: primewave_SPICE3
# File: simulator.tcl
namespace eval ::startup {
  set sim [sa::createSimulator SPICE3 -description "SPICE3" \
    -optionsFile [db::resolve primewave/SPICE3/options.xml]]
  db::setAttr netlistTemplate -of $sim -value SPICE3.template
  db::setAttr finalNetlist -of $sim -value input.spi
  db::setAttr logFile -of $sim -value "hspice.lis"
  db::setAttr includeTypes -of $sim -value [list \
    Options \
    Variables \
    Netlist \
    Analyses \
    Outputs \
    {Digital Vector}]
  ....
  ....
  ....
}
```

Refer to the SPICE3 integration for a complete example of the `simulator.tcl` file.

You must also define several callback routines for netlisting and simulation. For example, you can define a `runCommandProc` procedure, which gets the command-line arguments for running a simulator. The `createFinalNetlistProc` can be defined to write out the contents of the final netlist.

Refer to [Integration Objects and Callbacks](#) for a complete listing of possible Tcl callback routines that you can define. The callbacks are defined in the `sa::<simName>` namespace in the `<simName>.tcl` file.

Setting Up the startup.tcl File

A `startup.tcl` file is needed in each integration directory to source the files that define the integration.

The following code is a sample `startup.tcl` file:

```
primewave/SPICE3/startup.tcl

# Copyright (c) 2004-2015 Synopsys, Inc. This Custom Compiler
# software and the associated documentation are
# confidential and proprietary to Synopsys, Inc. Your use or
# disclosure of this Custom Compiler software is subject
# to the terms and conditions of a written license agreement
# between you, or your company, and Synopsys, Inc.
#
# DISCLAIMER
# The information contained herein is provided by Synopsys, Inc.
# on an "AS IS" basis without any warranty, and Synopsys has no
# obligation to support or otherwise maintain the information.
#
# Synopsys, Inc. disclaims any representation that the information
# does not infringe any intellectual property rights or proprietary
# rights of any third parties. There are no other warranties given
# by Synopsys, whether express, implied or statutory, including,
# without limitation, implied warranties of merchantability and
# fitness for a particular purpose.
#
# Synopsys, Inc. reserves the right to make changes to the
# information at any time and without notice.
#
#####
# Project:      sae_SPICE3
# File:         startup.tcl
# Description:  PrimeWave Design Environment SPICE3 package startup
#              script
package require window
package require primewave
db::init primewave/SPICE3/preferences.tcl
```

```
db::init primewave/SPICE3/simulator.tcl
db::init primewave/SPICE3/tran.tcl
db::init primewave/SPICE3/op.tcl
db::init primewave/SPICE3/dc.tcl
db::init primewave/SPICE3/ac.tcl
db::init primewave/SPICE3/SPICE3.tcl
db::init primewave/SPICE3/save_options.tcl
db::init primewave/SPICE3/environment_options.tcl
db::init primewave/SPICE3/pa_expressions.tcl
db::init primewave/SPICE3/param_map_utils.tcl
db::init primewave/SPICE3/language.tcl
xt::createJobLimit SPICE3 -defaultLimit 3
```

Setting Preferences

You can create integration-specific preferences that will be loaded and overridden as part of the standard PrimeWave Design Environment load sequence loads and overrides. Preferences can be created with the PrimeWave Design Environment and PrimeWave Design Environment <simulator> to save and load preferences as a group. Preferences can seed or save current field values in dialog boxes. All simulator-specific preferences must be defined in the `preferences.tcl` file, which is located in the simulator integration directory.

For example:

```
$SYNOPSIS_CUSTOM_SITE/primewave/SPICE3/preferences.tcl
```

Use the `db::createPref` command to create new preferences. You can also use the `sa::makePrefName` command to create preference names and the `sa::makePrefLabel` command to assign PrimeWave Design Environment labels. The `preferences.tcl` file must be sourced in the integration `startup.tcl` file using the `db::init` command.

The following code is an example of a `preferences.tcl` file:

```
primewave/SPICE3/preferences.tcl

# Copyright (c) 2004-2023 Synopsys, Inc. This Custom Compiler
# software and the associated documentation are
# confidential and proprietary to Synopsys, Inc. Your use or
# disclosure of this Custom Compiler software is subject
# to the terms and conditions of a written license agreement
# between you, or your company, and Synopsys, Inc.
#
# DISCLAIMER
# The information contained herein is provided by Synopsys, Inc.
# on an "AS IS" basis without any warranty, and Synopsys has no
# obligation to support or otherwise maintain the information.
#
# Synopsys, Inc. disclaims any representation that the information
# does not infringe any intellectual property rights or proprietary
```

```
# rights of any third parties. There are no other warranties given
# by Synopsys, whether express, implied or statutory, including,
# without limitation, implied warranties of merchantability and
# fitness for a particular purpose.
#
# Synopsys, Inc. reserves the right to make changes to the
# information at any time and without notice.
#
#####
# Project: primewave_SPICE3
# File: preferences.tcl
# Description: SPICE3 preference creation. Note that preferences
# relating to options are created at simulator registration time
# and not here.
db::createPref [sa::makePrefName "envOpts includePath" -simulator
  SPICE3] \
  -description "Search path for SPICE3 include files" \
  -labels [sa::makePrefLabel SPICE3] \
  -value "" -defaultScope "saTestbench"
db::createPref [sa::makePrefName "envOpts outputDir" -simulator SPICE3]
\
  -labels [sa::makePrefLabel SPICE3] \
  -description "Output Directory" \
  -value "" -defaultScope "saTestbench"
db::createPref [sa::makePrefName "models includePath" -simulator SPICE3]
\
  -description "Search path for SPICE3 model files" \
  -labels [sa::makePrefLabel SPICE3] \
  -value "" -defaultScope "saTestbench"
db::createPref [sa::makePrefName "envOpts switchViewList" -simulator
  SPICE3] \
  -description "Switch View List for netlister" \
  -labels [sa::makePrefLabel SPICE3] \
  -value [list hspice hspiceD cmos.sch cmos_sch schematic] \
  -defaultScope "saTestbench"
db::createPref [sa::makePrefName "envOpts stopViewList" -simulator
  SPICE3] \
  -description "Stop View List for netlister" \
  -labels [sa::makePrefLabel SPICE3] \
  -value [list hspice hspiceD] -defaultScope "saTestbench"
db::createPref [sa::makePrefName "envOpts commandLineOptions" -simulator
  SPICE3] \
  -description "Additional command line options for the SPICE3" \
  -labels [sa::makePrefLabel SPICE3] \
  -value "" -defaultScope "saTestbench"
db::createPref [sa::makePrefName "envOpts netlistLanguage" -simulator
  SPICE3] \
  -description "Netlist language for the SPICE3 simulator" \
  -labels [sa::makePrefLabel SPICE3] \
  -value "HSPICE"
```

In this example `preferences.tcl` file, several preferences are created that can be used to seed various fields in the PrimeWave Design Environment user interface when SPICE3 is the selected integration. For example, a preference is created in the environment options dialog box, which sets the switch view list.

Note:

The `netlistLanguage` preference is used to specify which netlister is used for the integration. In the SPICE3 example, the HSPICE netlister is used.

Creating Menus

The PrimeWave Design Environment creates core menus for the PrimeWave Design Environment window when invoked. These are common to all simulator integrations. The following core menus are included:

- Session
- Setup
- Variables
- Outputs
- Simulation
- Results
- Tools
- Windows
- Help

When you create a new integration, clones of all the core menus are created. You can retrieve these menus using the `gi::getMenu` command and then modify or add menus as needed.

Modifications and new menu definitions are specified in the `menus.tcl` file. Actions used for adding to cloned or new menus are also created in the `menus.tcl` file. You can also delete actions from cloned menus that are not needed for an integration.

Defining Shortcut Keys

You can define new shortcut keys (key bindings) in the `bindings.tcl` file. Shortcut keys are updated when you choose a simulator in the **Simulator** setup dialog box (**Setup > Simulator**) or when you activate a saved binding set. You might want to copy the

`bindings.tcl` file from the `<cd_installation_dir>/primewave/` installation directory and save it in the simulator integration directory for modification.

The following example creates a shortcut key for the **Edit Netlist** action:

```
set windowType [sa::getWindowType -simulator "SPICE3"]
gi::createBinding -windowType $windowType \
    -event 'shift-e' \
    -action saEditNetlist
```

For more information about customizing the binding set, see the *Custom Compiler Environment User Guide*.

For the list of the default PrimeWave Design Environment shortcut keys, see PrimeWave Design Environment Shortcut Keys.

Customizing Netlisters

Several netlisters are initialized when the PrimeWave Design Environment tool is invoked, and you can customize these netlisters for any simulation integration. Customized netlister code can reside in a file in the simulation integration directory, but it must be sourced in the simulator `startup.tcl` file using the `db::init` command.

You can customize the primary netlisters by modifying any of the following:

- netlister attributes
- netlisting callbacks
- device netlisting procedures

You can either create a copy of a primary netlister to modify or modify the actual netlister. If the primary netlister is modified directly, then any other part of the PrimeWave Design Environment tool that accesses the netlister also uses those modifications. For example, if the HSPICE netlister is modified directly, netlists generated through the Simulation Interface in the Schematic Editor or the Export Netlist menu in the console use the modified netlister.

Setting Netlisters for Integrations

To specify a netlister for your integration, create and set the `sa<simName>EnvOptsNetlistLanguage` preference in the `preference.tcl` file and initialize the environmentOptions "language" attribute to point to this preference. You can set the environmentOptions "Language" attribute in the `environment_options.tcl` file.

For example, in the SPICE3 integration, the netlist is set to HSPICE as follows:

```
db::createPref [sa::makePrefName "envOpts netlistLanguage" -simulator
SPICE3] \
  -description "Netlist language for the SPICE3" \
  -labels [sa::makePrefLabel SPICE3] \
  -value "HSPICE"
proc initialize {envOptions scope} {
  sa::initializeAttributes [getAttrs] -simulator HSPICE \
    -of $envOptions -scope $scope
  # initialize core attribute values from preferences
  foreach attr [list includePath switchViewList stopViewList
    commandLineOptions netlistLanguage] {
    set value [db::getPrefValue \
      [sa::makePrefName "envOpts $attr"
        -simulator SPICE3] -scope $scope]
    db::setAttr $attr -of $envOptions -value $value
  }
}
```

Defining Simulator Analyses

As part of new simulator integration, you can define analyses that a simulator supports. Each analysis must be defined in a separate Tcl file named after the analysis. For example, if a simulator supports only Transient (tran) and Operating Point (dc) analyses, you must define a `tran.tcl` and a `dc.tcl` file. Each file defines the analysis options.

You must include the `sa::createAnalysisDef` command in the startup namespace to create an `saAnalysisDef` object for each analysis type the simulator supports.

To perform various simulator functions, callbacks must also be defined in the `sa::<simulator>::<analysis_type>` namespace.

The following Tcl callback procedures are required:

- `createDialog` - Creates analysis specific fields. These appear in the **Analysis** dialog box when you choose **Setup > Analysis** and then the analysis type.
- `getAttrs` - Registers analysis field definitions such as:
 - field name
 - field type
 - field label
 - optional preference names for the analysis field
 - optional choices for the field values
- `initialize` - Initializes the fields defined in `getAttrs`.

- `getDisplayFields` - Returns the list of the analysis fields displayed when the analysis item is expanded in the Analyses pane of the main PrimeWave Design Environment window.
- `synopsis` - Returns the summary of analysis field values. This string value is displayed in the Analyses pane in the main PrimeWave Design Environment window when an analysis is collapsed in the tree.
- `getTests` - Returns the list of implemented test procedures used to validate the analysis field values that a user sets for this analysis type.

The following Tcl procedure callbacks are optional:

- `depends` - Checks the dependency between analyses.
- `supportsMultiple` - Returns true if multiple analyses of this type is supported.

See [Integration Objects and Callbacks](#) for more detailed information on these callbacks.

Refer to the SPICE3 integration for examples of ac, dc, tran, and op analyses definitions.

The following sample code shows how to use the `sa::createAnalysisDef` command, as well as some of the callbacks for a transient analysis:

```
# Copyright (c) 2004-2015 Synopsys, Inc. This Custom Compiler
# software and the associated documentation are
# confidential and proprietary to Synopsys, Inc. Your use or
# disclosure of this Custom Compiler software is subject
# to the terms and conditions of a written license agreement
# between you, or your company, and Synopsys, Inc.
#
# DISCLAIMER
# The information contained herein is provided by Synopsys, Inc.
# on an "AS IS" basis without any warranty, and Synopsys has no
# obligation to support or otherwise maintain the information.
#
# Synopsys, Inc. disclaims any representation that the information
# does not infringe any intellectual property rights or proprietary
# rights of any third parties. There are no other warranties given
# by Synopsys, whether express, implied or statutory, including,
# without limitation, implied warranties of merchantability and
# fitness for a particular purpose.
#
# Synopsys, Inc. reserves the right to make changes to the
# information at any time and without notice.
#
#####
# Project:      primewave_spice3
# File:        tran.tcl
# Description:  Support code for SPICE3 transient analysen
namespace eval ::sa::SPICE3::tran {
    # intiialize - create analysis attributes for transient analysin
```

```

# @param[in]  a analysin
# @return    a
proc initialize {a} {
    sa::initializeAttributes [getAttrs] -of $a -simulator SPICE3
    return $a
}
# getAttrs - get the attributes for this analysis type
# field definitons  fieldName type DialogLabel PrefName
enum values(if required)
proc getAttrs {} {
    <code to return field attributes>
}
# Construct a Transient Analysis dialog
# @param[in] parent -Parent window for the new dialog
# @return
proc createDialog {parent} {
    <code to create the dialog and various widgets>
}
#get the list of fields for display on the main console analysis pane
proc getDisplayFields {a} {
    <return list of display fields>
}
# get the list of validation tests to be run on this analysis
proc getTests {a} {
    <return list of check procedures>
}
# return synopsis text string for main analysis pane
# @param[in]  a analysis object
# @return    string
proc synopsis {a} {
    return [sa::showEditAnalyses::formatSynopsis SPICE3 $a [list
tstart tstep tstop]]
}
}
namespace eval ::startup {
    set sim [sa::getSimulators "SPICE3"]
    set newAnalysis 0
    if {![db::getCount [sa::getAnalysisDefs tran -of $sim]]} {
        set ad [sa::createAnalysisDef tran -title "Transient" -simulator
$sim]
    set newAnalysis 1
    }
    #create preferences
    if {$newAnalysis} {
        <create preferences>
    }
}
}

```

This section covers the following topics:

- [Creating Dialog Boxes and Registering Analysis Fields](#)
- [Creating Preferences for Analysis Fields](#)

- [Defining Selection Functions for Analysis Fields](#)
- [Validating Analysis Fields](#)
- [Defining Dependencies between Analyses](#)
- [Defining Synopsis Procedures for Analysis](#)
- [Defining Displayable Fields](#)

Creating Dialog Boxes and Registering Analysis Fields

To create the analysis dialog box and specify the analysis fields for each analysis type, define the `createDialog` callback. A core analysis dialog box is created, which you can access from the **Setup > Analysis** menu or by double-clicking an analysis in the analysis pane.

The core Analysis dialog box comes with the following features:

- A type selector widget for selecting the analysis type - The PrimeWave Design Environment core uses an `saSimulator` object to generate the list of analysis types that a simulator supports and creates a mutex input widget.

If the current analysis type supports multiples of the same type, the analysis dialog box displays widgets for selecting, creating, copying, and renaming analysis names.
- A parent frame widget - Creates the analysis-specific fields of the analysis object and corresponds to currently selected analysis names (in the name selector widget) and currently selected analysis types (in type selector widget). If the currently selected analysis type does not support multiples, the analysis title is used as the label for the parent frame widget. Otherwise, the concatenated text from the analysis title and the currently selected analysis name is used as the label. The label text is updated whenever the type selector or name widget is modified.
- Enable widget - Modifies the enable or disable status of the analysis object.
- Button Bar - Includes **OK**, **Apply**, **Cancel**, and other buttons. All buttons are created as part of the Opal dialog box.

The analysis selector widget is populated with the analysis types listed in the simulator object. The `createDialog` procedure is then called for each analysis type, and the analysis-specific fields are defined to populate the parent frame.

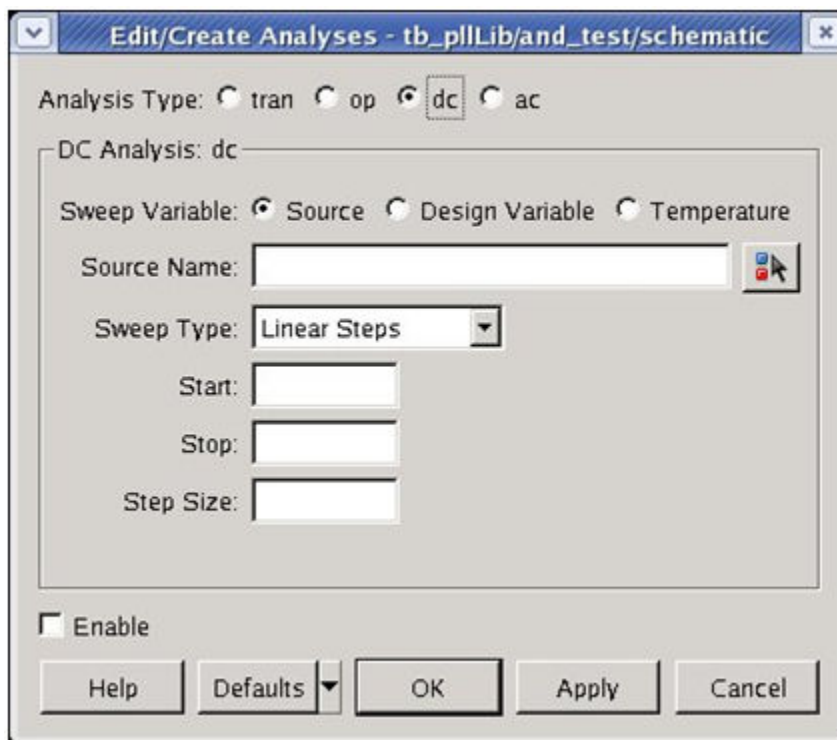
Refer to the *Tcl Reference Manual* for a list of `gi::create*` commands used to create dialog boxes and field inputs of various types.

The following example shows a `createDialog` procedure for a DC analysis:

```
proc createDialog {parent} {  
    set sweep [gi::createMutexInput sweep -parent $parent \
```

```
-label [sa::showEditAnalyses::getLabel SPICE3 dc sweep]\
    -value "Source" \
    -valueChangeProc
sa::SPICE3::dc::sweepVariableSelected \
    -prefName [sa::makePrefName "sweep" -simulator SPICE3 -analysis dc] \
    -enum [sa::showEditAnalyses::getEnumOptions SPICE3 dc sweep]]
#setup source picker
::sa::showEditAnalyses::setupDesignPick $parent SPICE3 dc source source
    false
#setup design variable picker
::sa::showEditAnalyses::setupDesVarPick $parent SPICE3 dc false
#sweep setup
set sweepOptions { {"Linear Steps" { start stop stepSize} } \
    {"Linear Points" {start stop numPoints } } \
    {"Octave" { start stop numPoints } } \
    {"Decade" { start stop numPoints } } \
    {"Points of Interest" { poi } } }
set sweepType [::sa::showEditAnalyses::setupSweep $parent SPICE3 dc
    sweepType $sweepOptions]
    sweepVariableSelected $sweep
    ::sa::showEditAnalyses::sweepTypeSelected
    $sweepOptions $sweepType
}
```

This code produces the following dialog box:



You must also define the following attributes for each analysis field:

- Analysis field name
- Analysis field type
- Label to be used for the analysis field
- (Optional) A preference name for the analysis field - If the generated preference name is not suitable for an analysis field, specify the preference name here. If the preference is created here, it does not need to be in the `preferences.tcl` file.
- (Optional) Choices for the field values - The list of items or values for the analysis field. If a list provided, a combobox is included in the analyses pane for the analysis field. See [Defining Displayable Fields](#) for more information on this.

You can use `getAttrs` callback to define analysis field attributes. The following `getAttrs` callback is an example of the DC analysis described previously:

```
# field definitons fieldName type DialogLabel PrefName enum_values(if
required)
proc getAttrs {} {
    set a {
        {sweep string {Sweep Variable} "" {"Source" "Design Variable"
"Temperature"}}
        {source      string {Source Name}}
        {designVar    string {Variable Name}}
        {sweepType   string {Sweep Type} "" {"Linear Steps" "Linear
Points" "Octave" "Decade" "Points of Interest"}}
        {start       string {Start}}
        {stop        string {Stop}}
        {stepSize    string {Step Size}}
        {poi         string {Points} saSPICE3DCPOI }
        {numPoints   string {No. of Points}}
    }
    return $a
}
```

Creating Preferences for Analysis Fields

You can define preferences for remembering the last values entered in a dialog box for each analysis field. Preference names are generated using the `sa::makePrefName` command. The preferences are registered at startup and are created in the startup namespace.

Defining Selection Functions for Analysis Fields

Analysis fields might require input from the design or design variables. Input from the design might be a single object or a pair of objects (although the input is usually nets). You

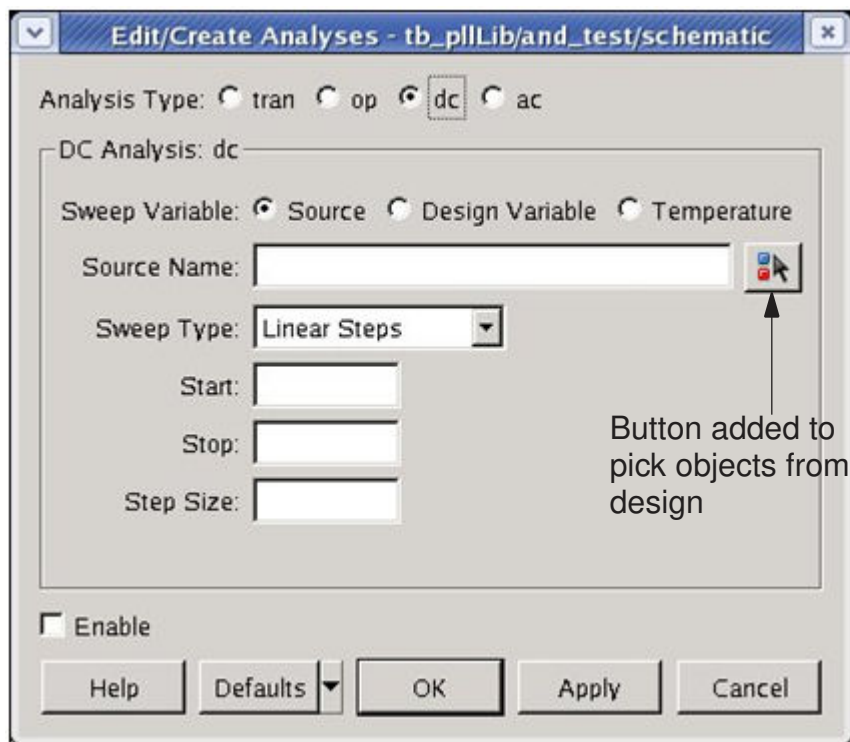
can define a button next to a field that is used for selecting an object in the design or a variable from the list of variables available in the PrimeWave Design Environment. Then, you can populate the field with the name of the object or variable.

The following functions are available:

Function	Argument	Description
sa::showEditAnalyses::setupDesignPick	dialog - The Edit/Create Analyses dialog object. simulatorName - The name of the simulator integration analysisType - The analysis type fieldname - Name of the field to which the button is attached. objectType - Can be set to instance, source, or net. show - If true, the field is displayed in the dialog box.	Adds a button for selecting an object in the design then populates the field with the name of the object.
sa::showEditAnalyses::setupDesVarPick	dialog - The Edit/Create Analyses dialog object simulatorName - The name of the simulator integration. analysisType - The analysis type. fieldname - Name of the field to which the button is attached. show - if true, the field is displayed in the dialog box.	Adds a button for selecting a variable from a list of variables that exist in a PrimeWave Design Environment session.
sa::showEditAnalyses::setupNodePairPick	dialog - The Edit/Create Analyses dialog object. simulatorName - The name of the simulator integration. analysisType - The analysis type. fieldname - Name of the field to which the button is attached. show - If true, the field is displayed in the dialog box.	Adds a button for selecting a pair of nodes in the design and then populates the field with the name of the nodes.

The following example adds a button to pick an object of "source" type in a design to the **Source** field for a DC analysis in the SPICE3 integration:

```
::sa::showEditAnalyses::setupDesignPick $parent SPICE3 dc source source
false
```



Validating Analysis Fields

You can define procedures to validate the dialog box appearance and entered values in fields and analysis setup.

The first type of procedure validates the appearance of the analysis setup dialog box when widgets are enabled, disabled, added, or removed according to what the user chooses in a dialog box. These procedures are set as `valueChangeProc` arguments for each command that creates a widget.

For example, in a particular analysis type, a sweep type selection with the LINEAR, LOGARITHMIC, or Points of Interest choices might be available. If the user chooses LINEAR, the dialog box needs to display a start field, a stop field, and a step field. If the user chooses Points of Interest, the dialog box needs to display a text field so the user can enter a list of numbers. In this case, you can define a procedure that validates the dialog box when the SWEEP type and resulting dialog box fields change.

The second type of procedure validates the values entered in various fields of the analysis setup, which ensures the analysis setup is correct when a user clicks **OK** or **Apply**. When you create a validation procedure for each field, the procedure is executed when a user clicks **OK** or **Apply**. The `getTests` procedure must be implemented so that a list of validation procedures that must be executed is returned.

Several checking functions are available for use in the validation procedures. These functions return an error message that is displayed in a ToolTip on the field that contains an error.

For more information on error checking functions, refer to the [Integration Objects and Callbacks](#).

Defining Dependencies between Analyses

You can indicate analysis types that have dependencies on other analysis types by defining the `sa::<simulator>::<analysis_type>::depends` Tcl callback procedure. For example, if an AC analysis must be defined for a noise analysis to work, include a `depends` procedure for the noise analysis definition. The callback should do the following:

If the dependency check...	then...
passes	return zero (0)
fails	return one (1) and call the <code>sa::check::createWarning</code> procedure with the warning message text to be used for reporting the dependency check failure.

Dependency procedures are called after the validation procedures run. If the check fails, the dialog box is not dismissed, and the first failed analysis is displayed with the warning message at the bottom of the form. The following example shows how you can define the `depends` procedure for an FFT analysis, which needs to have the transient analysis enabled:

```
# this analysis depends on having an enabled Tran analysis
proc depends {gui tb a} {
    db::foreach ana [sa::getAnalyses -of $tb] {
        set def [db::getAttr definition -of $ana]
        if { $def == "tran" } {
            if { [db::getAttr enabled -of $ana] == 1 } {
                return 1
            }
        }
    }
    set warning "Warning: A Tran Analysis must be enabled"
    sa::check::createWarning $gui $tb $a $warning
    return 0
}
```

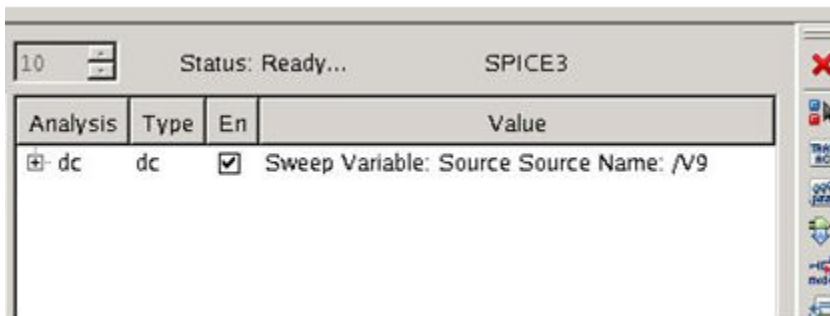
Defining Synopsis Procedures for Analysis

The `sa::<simulator>::<analysis_type>::synopsis` Tcl procedure returns the summary of the analysis fields, which is displayed in the analysis pane of the PrimeWave Design Environment window when the analysis is collapsed.

The following sample shows a synopsis procedure for a DC analysis:

```
proc synopsis {a} {
    set sweep [db::getAttr sweep -of $a]
    switch -exact $sweep {
        "Source" {
            return [sa::showEditAnalyses::formatSynopsis SPICE3
$a [list sweep source]]
        }
        "Design Variable" {
            return [sa::showEditAnalyses::formatSynopsis SPICE3
$a [list sweep designVar]]
        }
        "Temperature" {
            return [sa::showEditAnalyses::formatSynopsis SPICE3
$a sweep]
        }
    }
}
```

The following image shows the Analysis pane in the PrimeWave Design Environment with a DC sweep:



Defining Displayable Fields

For each analysis type, you must specify which fields can be displayed in the Analysis pane of the PrimeWave Design Environment window via the `sa::<simulator>::<analysis_type>::getDisplayFields` procedure. The PrimeWave Design Environment uses `getAttrs` to get the field types, which determines the widget

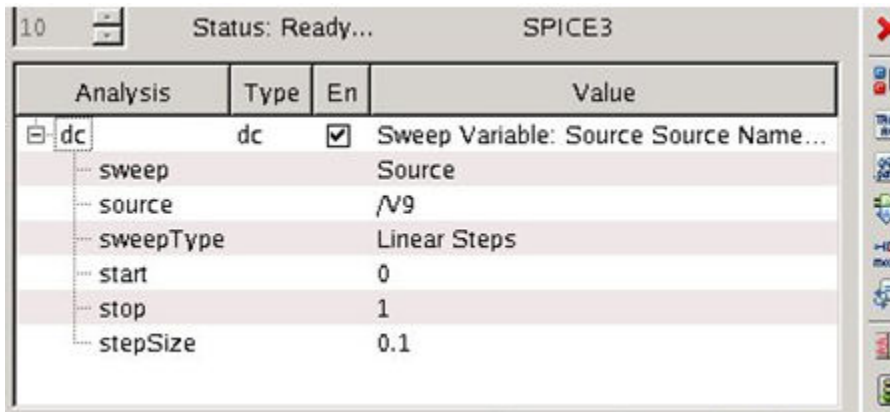
that displays it in the Analysis pane. The following table shows the widget type used for different field types:

Field Definition		Widget Type
Type	Choices	
boolean	True/false	Check Box
string	Empty String or 1 item	Text Edit
string	Two or more items	Combo Box

The following sample `getDisplayFields` procedure is for a DC analysis:

```
proc getDisplayFields {a} {
    set fields {sweep}
    switch -exact [db::getAttr sweep -of $a] {
        "Source" {
            lappend fields "source"
        }
        "Design Variable" {
            lappend fields "designVar"
        }
    }
    lappend fields sweepType
    set sweepOptions { {"Linear Steps" { start stop stepSize} } \
        {"Linear Points" {start stop numPoints } } \
        {"Octave" { start stop numPoints } } \
        {"Decade" { start stop numPoints } } \
        {"Points of Interest" { poi } } }
    foreach option $sweepOptions {
        set sel [lindex $option 0]
        if { $sel == [db::getAttr sweepType -of $a] } {
            set fieldList [lindex $option 1]
            foreach field $fieldList {
                lappend fields $field
            }
        }
    }
    return $fields
}
```

The following Analysis pane in the main PrimeWave Design Environment window shows when a DC sweep is expanded:



Customizing Dialog Boxes

You can customize the save options and environment options dialog boxes to add additional fields. The following procedures must be defined for each dialog box that you want to customize:

- `sa::<simulator>::<dialog_box_name>::createDialog`
Creates the additional fields for the dialog.
- `sa::<simulator>::<dialog_box_name>::getAttrs`
Defines the field attributes.
- `sa::<simulator>::<dialog_box_name>::validate`
Validates the entries into the fields.

You must also define how the data entered into these fields is used.

For example, in the SPICE3 integration, an environment option to use a 64-bit simulation is added. The setting affects the command that is used to launch the simulation.

In the `createDialog` procedure, the following code is defined:

```
gi::createBooleanInput use64Bit \
    -label "64-bit simulation" \
    -prefName [sa::makePrefName "envOpts use64Bit" \
    -simulator SPICE3] \
    -value [db::getAttr use64Bit -of $envOptions] \
    -parent $parent
```

In the `SPICE3.tcl` file, the following code is added to the command string built in the `runCommand` procedure:

```
if {[expr("[sa::utils::getUnixEnvVariableValue HSPICE_64]" \
    == 1)]}{
    set envOpts [db::getAttr environmentOptions -of $tb]
    if {[db::getAttr use64Bit -of $envOpts]}{
        append command "unset HSPICE_64; "
    }
}
```

Creating New Dialog Boxes

New dialog boxes can be created using any of the `gi::` commands to add to the integration. These dialog boxes may be tied to new menus added in the PrimeWave Design Environment window. You should create an additional attribute in the testbench object to hold the data entered in the dialog box. You must also specify how the data is used in creating the simulator input file.

Specifying Simulator Options

You can create an XML file to define simulator options, which the user can access through the simulator options GUI. This GUI is accessible from **Simulations > Options**.

The following attributes must be defined in the `options.xml` file for each option:

Option	Description
<code>name</code>	The name of the option. This is the name that will be presented on the GUI.
<code>type</code>	The option value type. This can be int, float, string or Boolean.

The following optional attributes can be defined for each simulator option:

Option	Description
<code>prompt</code>	A short label that can be displayed instead of the name.
<code>defaultValue</code>	The default value used by the simulator if the option is not set.
<code>pref</code>	The unique name for the preference, which is used to store the option value. If <code>pref</code> is not specified, a generated name is used.
<code>shortDescription</code>	A brief description of the option, which can be displayed in a tooltip when the mouse hovers over the option in the options GUI.

Option	Description
description	A description that is printed on the PrimeWave Design Environment console window when the user selects Description from the context-sensitive menu.
supportsExpression	For int or float values, setting this to true indicates that a variable or expression can be set as the option value.
enable	If set to true, then an Enable checkbox appears in the value column for the simulator option and must be selected for the simulator option to be netlisted. The <code>enable</code> option is used for simulator options that have no effect unless they appear in the netlist.
conditionalDefaults	Specifies if the default value for an option depends on other options. This can also be used to indicate if the inherited value is then editable or ignored by the simulator.
enum	Specifies the permitted enumerated list of values for an option.
min	Specifies a lower limit when the value type is int or float and is automatically enforced.
max	Specifies an upper limit when the value type is int or float and is automatically enforced.
help	Specifies the path to the help document for the simulator relative to the simulator install directory.

The following is an example options file for the SPICE3 integration:

```
<?xml version="1.0"?>
<simulator>
<comment>
This is a sample options definition file for SPICE3
</comment>
<optionDef name="ABSTOL" type="float">
  <defaultValue>1e-12</defaultValue>
  <description>Resets the absolute current error tolerance of the
program.</description>
</optionDef>
<optionDef name="METHOD" type="string">
  <defaultValue>TRAPEZOIDAL</defaultValue>
  <description>Sets the numerical integration method used by
SPICE.</description>
  <limit type="enum">
    <value prompt="GEAR">GEAR</value>
    <value prompt="TRAPEZOIDAL">TRAPEZOIDAL</value>
  </limit>
</optionDef>
</simulator>
```

You can define categories in the simulator.tcl file, which can be used to filter the options displayed in the GUI.

The following example file shows categories created for the options in the SPICE3 integration, as well as how a text field can be created in the options dialog box for entering text freely.

```
set topGroup [db::getAttr optionGroup -of $sim]
set od [sa::createOptionDef "otherOptions" -prompt "Other Options"
      -optionType "string" -simulator $sim \
      -description "For including simulator options not
supported by this GUI (format is name1=val1 name2=val2 ...)"]
sa::createOptionGroup "Other Options" \
      -description "For including options not supported by this GUI" \
      -optionNames [list "otherOptions"] \
      -in $topGroup
sa::createOptionGroup "All" -optionNames $names -in $topGroup
```

Refer to the [Integration Objects and Callbacks](#) for the XML schema definition which describes the format and structure of the simulator options.

Creating Simulator Input Files

The simulator input file is created when the user invokes the netlisting command or a simulation command. The simulator input file is created as defined by the control template using the structural netlist and the data setup in the PrimeWave Design Environment session. You must define the `sa::<simulator>::createFinalNetlistProc` in the `<simulator>.tcl` file to write data from the PrimeWave Design Environment session to respective files in the run directory. The control template can include all the possible files as well as the structural netlist. The final netlist is created using the structure defined in the control template. The user might instead explicitly define how the final netlist is written and leave out the control template definition in the `createFinalNetlistProc`.

The following sample is a `createFinalNetlistProc` function used in the SPICE3 integration:

```
proc createFinalNetlistProc {tb} {
    writeHeader $tb
    writeOptionsSearch $tb
    writeOutputFormat $tb
        writeOptions $tb
        writeModelFiles $tb
        writeVariables $tb
        writeConvergence $tb
        writeStimulusGenerator $tb
        writeAnalyses $tb
        writeOutputs $tb
        writeTemperature $tb
}
```

```
        writeVariationOptions $tb  
    }
```

Refer to the `SPICE3.tcl` file for the `write*` procedures. The following netlist template file is for the SPICE3 integration:

```
[n?.netlistHeader]  
[n?.optionsSearch]  
[n?.variation]  
[n?.designVariables]  
[n?.options]  
[n?.optionsFormat]  
[n?.temperature]  
[n?.modelFiles]  
[!netlist]  
[n?.convergence]  
[n?.stimulusGenerator]  
[n?.analyses]  
[n?.outputs]  
[n?.optionsBiasChk]  
[n?.alterBlock]  
.end
```

See the [Integration Objects and Callbacks](#) for descriptions of the various markups that you can use in the template file.

The `postFinalNetlistProc` procedure can also be defined to further process the final simulator input file before launching the simulation.

Defining Simulator Run Commands

A Tcl callback must be defined to build the simulation run command. The procedure is called when the user invokes a simulation and writes the command to the `runSimulation` file, which is located in the netlist directory. The run command usually contains the simulator executable, command line options, and an optional command to redirect the output to a log file. The log file is specified in the `logFile` attribute of the simulator object.

You can use the `sa::<simulator>::runScriptProc` and the `sa::<simulator>::runCommandProc` commands for generating the command for simulation. The `runScriptProc` is called when the `runSimulation` file is created. The `runSimulation` file contains the command for running the simulations manually from a UNIX prompt. The `runCommandProc` is called when a command is required for launching a simulation. These must be defined in the `<simulator>.tcl` file.

Defining Post-Processed Simulation Results

You must define how the simulator output format can be interpreted, plotted, and post-processed to evaluate expressions set up by the user. The following tasks need to be defined:

Tasks	Description
Mapping Data	Maps the names of signals in the waveform files between the simulator native output namespace and the PrimeWave Design Environment namespace.
Defining Results Maps for Simulator Output Files	Contains information mapping simulator results files to analysis information set up in the PrimeWave Design Environment, as well as location of results directories and netlist directories.
Mapping Operating Point Parameter Name	Maps names used in the simulator operating point results to names used in the opPointLabelSet in the CDF of a device.

Mapping Data

Data mapping between the simulator namespace and the PrimeWave Design Environment namespace needs to be defined for cases where the netlister name mapping is not sufficient to translate the name of a signal from one namespace to another. For example, in HSPICE, the current from the drain of a MOSFET is labeled `i1(...)`. This information is not available in the netlister name mapping and must be provided so that users can plot or evaluate MOSFET drain currents.

The `sa::getData` data access function maps the signal name from the PrimeWave Design Environment namespace to the netlister namespace. The `dataMap` function then gets the actual signal name that can be plotted (for example, `i1(m1)` for the current through the drain of `m1`).

You must define the `dataMap` Tcl procedure in the `sa::<simulator>::<resultType>` namespace. The signature of this procedure has the following string arguments:

- **Mapped Signal Name** - The signal name returned from the netlister mapping functionality.
- **Parameter Name** - The value of the `-param` argument is passed to the `sa::getData` as a parameter name to the `dataMap` procedure.
- **Data Type** - The value of the `-type` argument is passed to the `sa::getData` as the data type to the `dataMap` procedure. This can include voltage, current, power and any other signal types that can be returned for an analysis.

- Path to the Netlist Directory - The path to the netlist directory.
- Output format - The analysis output file format.

The following example shows the `dataMap` function used for a current signal:

```
proc dataMap {name dataType netlistDir format} {  
    return [sa::SPICE3::mapSimulatorName $name $dataType $netlistDir  
        $format]}  
proc mapSimulatorName {name dataType netlistDir format} {  
    switch $dataType {  
        voltage {return [sa::SPICE3::mapVoltageSignal $name $netlistDir]}  
        current {return [sa::SPICE3::mapCurrentSignal $name $netlistDir]}  
        power {return [sa::SPICE3::mapPowerSignal $name $netlistDir]}  
        totalPower {return [sa::SPICE3::mapTotalPowerSignal]}  
        capacitance {return [sa::SPICE3::mapCapSignal $name $netlistDir]}  
        default {return ""}  
    }  
}  
proc mapCurrentSignal {name netlistDir } {  
    array set data {}  
    ::sa::SPICE3::language::getTerminalPlotInfo $name $netlistDir data  
    return [formatTerminalPlot data]  
}  
# Format the current signal name for plotting  
proc formatTerminalPlot {argsArr} {  
    upvar $argsArr data  
    set str ""  
    switch $data(subType) {  
        subcktCurrent {set str  
            "$data(path).isub($data(terminalName))"}  
        deviceCurrent {  
            if { $data(path) == "" } {  
                set str "i($data(primitiveName))"  
            } else {  
                set str "$data(path).i($data(primitiveName))"  
            }  
        }  
        branchCurrent {  
            if { $data(path) == "" } {  
                set str "i$($data(index))\($data(primitiveName))"  
            } else {  
                set str  
                    "$data(path).i$($data(index))\($data(primitiveName))"  
            }  
        }  
        verilogaCurrent {  
            if { $data(path) == "" } {  
                set str "i($data(primitiveName)).$data(terminalName)"  
            } else {  
                set str  
                    "i($data(path)).$data(primitiveName).$data(terminalName)"  
            }  
        }  
    }  
}
```

```
    }  
  }  
  return $str  
}
```

Defining Results Maps for Simulator Output Files

The results map is a set of PrimeWave Design Environment core objects that represent information about simulator output files, relationships between analyses, analysis types, and output files. The location of simulation data, including netlist and results directories, is also in the results map. You must define the result map objects using the PrimeWave Design Environment command interface.

Results maps provide the following functions:

- Mapping between analysis types and output file names so that the correct signal is plotted in the waveform viewer.
- Mapping between analysis names (when multiple analyses of the same type exist) and the output file name.
- Back-annotating operating point information when the operating point results appear in multiple files.
- Executing `results` and `selectResult` commands, which need information about the results available in the results directory.
- Enabling and disabling PrimeWave Design Environment menus that depend on the presence of certain results. For example, **Results > Plot Signal > Transient** is enabled if the output file that corresponds to the transient analysis is available.

You must define the `sa::<simulator>::createResultsMapProc` command in the `<simName>.tcl` file with the following arguments:

- A collection of `saAnalysisInfo` objects (one for each enabled analysis). Each `saAnalysisInfo` object contains the analysis name, the analysis type, and the index of the analysis name with respect to the analysis type. These are automatically created.
- A testbench object. This object contains the setup information used for the simulation. This temporary testbench object is created using the state files saved in the results directory. The state information in the results directory is used because the information in a PrimeWave Design Environment session might not match the results.
- A `saResults` object. Add the mapping information about each output file to this object by calling the `sa::addResultFile` API.

Use the `saAnalysisInfo` and `testbench` objects to update the `saResults` objects in the `createResultsMapProc` with the following information:

- Get the names of all the output files found in the simulation run directory.
- Filter the output files which require mapping information. You can use `{saAnalysisInfo}` objects for this purpose. You can use the `testbench` object for getting additional information required for filtering the output files.
- Find the mapping information for each output file. You can use `{saAnalysisInfo}` objects and the `testbench` object.
- Call the API `sa::addResultFile` for updating the `saResults` object with mapping information for each output file.

Refer to the SPICE3 integration for an example of the `createResultsMapProc` function.

Refer to the SPICE3 integration for some examples.

Mapping Operating Point Parameter Name

The annotation of the device operating points to the schematic is controlled using the `opPointLabelSet` property in the cell CDF. The annotation feature in the PrimeWave Design Environment uses this property in the CDF cell for all simulators, but some simulators might use parameter names in the simulation output file, which are different from the names specified in the `opPointLabelSet` field for a device. The `op` parameter name mapping takes care of variations in the parameter names across simulators.

The Device Label Editor in the Custom Compiler tool displays all the OP parameter names for a selected device. You can select a different parameter not listed in the `opPointLabelSet` property for annotating a device operating point. If a simulator supports multiple output formats and uses different names for the operating points, the OP parameter name mapping makes the differences transparent by displaying the same parameter names for a device in the Device Label Editor.

You can define the `sa::<simulator>::mapOpPointParamProc` Tcl function with the following arguments:

- Instance name (`<string>`) - The instance name as well as the full occurrence path in simulator namespace of the device.
- Parameters (`<string>`) - One of the `op` parameter names in the `op` output file for the instance name.
- Netlist directory (`<string>`) - The netlist directory.
- Format (`<string>`) - The file format of the `op` output file.

The procedure must return the mapped name of the given parameter for the given instance.

The following example is a `mapOpPointParamProc` procedure as defined in the SPICE3 integration:

```
proc mapOpPointParamProc {instance param netlistDir format } {  
    return [sa::SPICE3::paramMapUtils::mapOpPointParam  
        $instance $param $format]}
```

Customizing the Results Analyzer

The Results Analyzer in the PrimeWave Design Environment provides both manual and automated expression building and plotting. The automated feature (the Plot Assist tab) generates and plots common measurements without concern for the actual syntax of the expression. In addition, the user can choose exactly how and what types of design signals they want to use to construct the final expression. The expressions available from the Plot Assist tab are filtered according to both the integrated simulator and the simulation result type selected (in the Analysis pane).

The Results Analyzer must be registered according to the simulator and result type for all expressions that the Plot Assist tab supports. Due to commonality across simulators, functions are available for registering basic expressions and associated signal selection options for AC, DC, and Transient analyses.

The following example shows how AC analysis expressions are registered:

```
::sa::plotAssist::registerAcVoltageExpressions SPICE3  
::sa::plotAssist::registerAcCurrentExpressions SPICE3  
::sa::plotAssist::registerAcSignalExpressions SPICE3
```

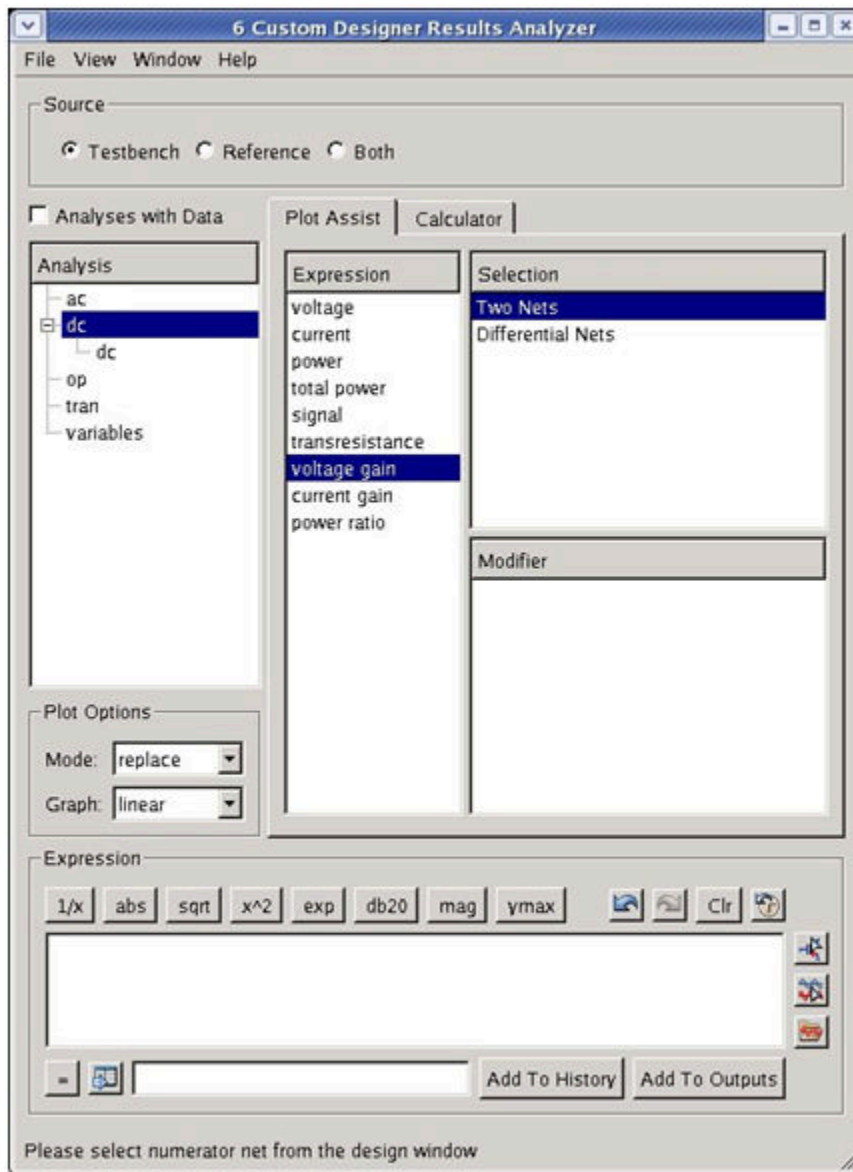
More complex expression generators can also be registered. Users must create an `saExpressionDef` object for each expression needed. Users can define `saSelectionDef` objects for each expression to add different selection modes to the Plot Assist Selection pane. An `saSelectionGroupDef` can be created to group `saSelectionDef` objects in one item, which allows users to define expressions that require multiple inputs from the design.

The following example shows how a voltage gain expression is created for a DC analysis. Two selection modes are provided: one requires selecting two design signals, and another requires four.

```
# voltage gain expression  
set ad [sa::getAnalysisDefs dc -of [sa::getSimulators SPICE3]]  
set ed [sa::createExpressionDef "voltage gain" -analysisDef $ad \  
    -createExpressionProc  
    "::sa::SPICE3::createDcVoltageGainExpression"  
    set sgdc [sa::createSelectionGroupDef twoNets -in $ed \  
        -label "Two Nets"]
```

```
sa::createSelectionDef s0 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "numerator net"
sa::createSelectionDef s1 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "denominator net"
    set sgd [sa::createSelectionGroupDef twoDiffNets -in $sed \
        -label "Differential Nets"]
sa::createSelectionDef s0 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "numerator primary net"
sa::createSelectionDef s1 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "numerator reference
net"
sa::createSelectionDef s2 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "denominator primary
net"
sa::createSelectionDef s3 -objectTypes { net instTerm terminal } -in
$sgd \
    -signalType net -prompt "denominator reference
net"
```

The following figure shows the Results Analyzer with the voltage gain expression selected:



You can register expressions in the Results Analyzer using the `pa_expressions.tcl` file, but the registration functions must be called in the `::startup` namespace. Refer to the [Integration Objects and Callbacks](#) for functions that customize the Results Analyzer.

Customizing Existing Integrations

Any of the integrations located in the installation directory can be customized at the SITE, PROJECT, USER, or LOCAL levels. You can create a directory in the same name as the integration to be modified and define a procedure to override the ones defined in the installation directory. You can also change or add menus, preferences, and callbacks by following the structure detailed in this chapter.

A

Integration Objects and Callbacks

This appendix contains a description of PrimeWave Design Environment objects and callbacks.

This appendix covers the following topics:

- [Objects](#)
- [Callbacks](#)
- [Error Checking Procedures](#)
- [Results Analyzer Registration Functions](#)
- [Options XML Schema Definition File](#)
- [Using Markup Syntax](#)

Objects

The following is the list of PrimeWave Design Environment objects:

- [saSimulator](#)
- [saAnalysisDef](#)
- [saAnalysisInfo](#)
- [saResultsMap](#)
- [saResultsInfo](#)
- [saResults](#)
- [saResultFile](#)
- [saSelectionDef](#)
- [saSelectionGroup](#)
- [saSignalObject](#)

- [saSelection](#)
- [saExpressionDefProcArgs](#)
- [saExpressionDef](#)

saSimulator

Description

The saSimulator object is created for each simulator integrated into PrimeWave Design Environment. The saSimulator object for any integration can be retrieved using the `sa::getSimulators` command.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	The name of the simulator.
displayName	<string>	read/write	Display name for the simulator.
finalNetlist	<string>	read/write	The name of the final netlist file. The default value is finalNetlist.
structuralNetlist	<string>	read/write	The name of the structural netlist. The default value is netlist.
languages	{<string>}	read/write	Collection netlist formats (languages). The default netlist format is <simulator_name>.
includeTypes	{<string>}	read/write	The external file types supported in the Include Files dialog box for the simulator integration.
logFile	<string>	read/write	The name of the simulator log file. The default log file name is <simulator_name>.out.
netlistTemplate	<string>	read/write	Name of the template file containing the overall structure of the final netlist. The include files in the template are specified relative to the netlist directory.
analysisDefs	{<saAnalysisDef>}	read/write	Collection of saAnalysisDef objects.
integrationVersion	<int>	read-only	The field the integration version can be specified.
optionsDef	{<saOptions>}	read/write	Collection of saOptions.
optionGroup	<saOptionGroup>	read/write	The saOptionGroup.

saAnalysisDef

Description

The saAnalysisDef object is created for each analysis type supported by the simulator. `sa::createAnalysisDef` can be used to create this object.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name of the defined analysis type such as tran and DC.
simulator	<string>	read-only	Name of the simulator.
title	<string>	read/write	Title for the analysis. For example, "Transient Analysis" for the analysis type "tran".
optionDefs	<saOptionDef>	read/write	Collection of saOptionDef.
optionGroup	<saOptionGroup>	read/write	The saOptionGroup object.
expressionDefinitions	<saExpressionDef>	read/write	Collection of saExpressionDef.

saAnalysisInfo

Description

The saAnalysisInfo object is a transient object created by the PrimeWave Design Environment for each enabled analysis and passed to the Tcl callback procedure `sa::<simulator>::createResultsMapProc` for creating the results map files.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	The name of the analysis.
analysisType	<string>	read-only	The defined analysis type.
index	<int>	read-only	The analysis number with respect to analysis type.

saResultsMap

Description

The saResultsMap object is created for each testbench at the end of a successful simulation run. `sa::createResultsMap` and `sa::openResultsMap` both return an saResultsMap object.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	The resultsMap.xml file.
resultsInfo	{<saResultsInfo>}	read/write	Collection of saResultsInfo objects.
timestamp	<string>	read/write	Creation time of resultsMap.xml.
resultsMapDir	<string>	read/write	The directory where resultsMap.xml is created.
simulator	<string>	read/write	Name of the simulation that generated the results.

saResultsInfo

Description

The saResultsInfo object is created for each simulation run. For nominal simulation, only one instance of saResultsInfo is created. For corners, parametric, and statistical analysis, multiple saResultsInfo objects are created. This can be retrieved from the saResultsMap object using the `db::getAttr` command.

Attributes

Name	Type	Read/Write	Description
netlistDir	<string>	read/write	The netlist directory.
resultsDir	<string>	read/write	The results directory.
params	{<saPair>}	read/write	Collection of saPair objects. This attribute is created only for parametric simulation. An saPair object is created for each sweep parameter.

saResults

Description

The saResults object is created at the end of a (successful or failed) simulation run. This can be retrieved using the `sa::getResults` command.

Note:

The parametric, corners, and statistical analysis tools can run multiple simulations. In this case, multiple saResults objects are present.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	The results.xml file.
version	<string>	read-only	Version number.
simulator	<string>	read/write	Simulator name.
netlistDir	<string>	read/write	The netlist directory.
resultsDir	<string>	read/write	The simulation result directory where the simulation output files are created.
runTime	<string>	read/write	The simulation completion date and time.
resultFiles	{<saResultFile>}	read/write	The collection of saResultFile object.

saResultFile

Description

The saResultFile object is created for each simulation output file. This can be retrieved using the `sa::getResultFiles` command.

Attributes

Name	Type	Read/Write	Description
filename	<string>	read/write	Simulation output file name.
resultType	<string>	read/write	Result type such as tran, opBegin, and opTran.
analysisName	<string>	read/write	Analysis name such as tranA and tranB.
format	<string>	read/write	Output file format such as .psf or .wdf.

saSelectionDef

Description

The saSelectionDef object encompasses the information needed to make one signal selection, which is when a user select something on a design and it is translated into a signal or signals. This can be created using the `sa::createSelectionDef` command.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name of the selection def object. Must be unique in an saSelectionDefGroup object.
label	<string>	read/write	Description of the signal being selected.
prompt	<string>	read/write	Description of the object to prompt selection of in design window.
objectTypes	[<enSelectionObjectType>]	read/write	List of object types allowed to pick from the schematic.
extractedSignalType	<enSelectionDefSignalType>	read/write	Type of extracted signal that is created from a picked object.
tclFilter	<string>	read/write	Name of Tcl procedure used to filter objects during picking from the schematic.
objectFilters	<string>	read/write	List of built in object filters allowed for this selection.

saSelectionGroup

Description

The saSelectionGroup object defines a sequence of signal selections required to create an expression. This can be created using the `sa::createSelectionGroup` command.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name identifying group of signals. Unique in an saExpressionDef object.
label	<string>	read/write	Small description that is displayed in the Plot_Assist Selection list.

Name	Type	Read/Write	Description
selectionDefs	{<saSelectionDef>}	read/write	The ordered list of saSelectionDef objects, which is requested from the user when this selection method is chosen.

saSignalObject

Description

The saSignalObject object represents a signal derived from an object the user has selected from the design. These extracted signals are passed to a filterProc procedure, which can modify them before they are sent to the createExpressionProc procedure.

The `saExpressionDef::createExpressionProc` procedure commonly uses the list of extracted signals for creating the expression.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name identifying the signal object.
objType	<enSignalObjectType>	read-only	Type of signal.
parentTerminal	<string>	read-only	Name of optional co-dependent terminal signal (including hierarchy).
useParentTerminal	<bool>	read/write	Flag indicating whether parent terminal is used.
outOfContext	<bool>	read-only	Flag indicating if the signal is out of context.

saSelection

Description

The saSelection object represents the actual selection a user makes during `ide::pick`, which saExpressionDef Tcl procedures can use.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name identifying selection object.
objectName	<string>	read/write	Name of the OA object selected (during <code>ide::pick</code>).

Name	Type	Read/Write	Description
objectType	<enSelectionObjectType>	read-only	Type of object selected (during ide::pick).
context	<deContext>	read/write	Design context.
selectionDefinition	<saSelectionDef>	read-only	saSelectionDef defining the type of selection this represents.
extractedSignals	{<saExtractedSignal>}	read-only	Signals derived from object based on methodology given by the selectionDefinition.
outOfContext	<bool>	read/write	Flag to indicate if the pick is out of context.
oneShot	<bool>	read/write	
inFilterLoop	<bool>	read/write	
onlySingleBitName	<bool>	read/write	
instTerm	<bool>	read/write	

saExpressionDefProcArgs

Description

The saExpressionDefProcArgs is an extendable core object, which is used as a container for passing arguments to the callback procedures that saExpressionDef names.

Note:

A few of the attributes do not exist for all callbacks.

Attributes

Name	Type	Read/Write	Description
expressionDefName	<string>	read-only	Name of expressionDef calling the proc. Exists for all callbacks.
analysisDefName	<string>	read-only	Name of Analysis Definition. Exists for all callbacks.
resultType	<string>	read-only	Name of result type chosen in Results Analyzer. Exists for all callbacks.
includeResultType	<bool>	read-only	Flag indicating if resultType should be explicitly used in expression. Exists for createExpressionProc callback only.

Name	Type	Read/Write	Description
analysisName	<string>	read-only	Name of an actual analysis. Exists for all callbacks.
includeAnalysisName	<bool>	read-only	Flag indicating if analysis name should be explicitly used in expression. Exists for createExpressionProc callback only.
opalPane	<giGroup>	read-only	Parent group widget where custom widgets can be created and used. Exists for all callbacks.
resultDir	<string>	read-only	Result directory. Exists for all callbacks.
includeResultDir	<bool>	read-only	Flag indicating if result directory should be explicitly used in expression. Exists for createExpressionProc callback only.
selectionGroupName	<string>	read/write	Name of saSelectionDefGroup in associated saExpressionDef. Exists for createExpressionProc only.
selections	{<saSelection>}	read-only	Collection of selection objects containing signal selection data. Exists for createExpressionProc only.

saExpressionDef

Description

The saExpressionDef object is the definition of an entry into the Plotting Assistant's expression list. This object contains the plans to generate a family of expressions for one particular simulator, analysis, or expression. Use the `sa::createExpressionDef` command to create this object.

Attributes

Name	Type	Read/Write	Description
name	<string>	read-only	Name (usually of a function) that identifies the set of expressions that can be generated. Uniquely identifies a PAFFunction.
label	<string>	read/write	Label that is displayed in the Plot Assist tab in the Expression pane.
description	<string>	read/write	Description of the expression(s) that are defined by this expression builder.
selectionGroupDefs	{<saSelectionGroup Defs>}	read/write	Collection of valid selection methods for this saExpressionDef.

Name	Type	Read/Write	Description
setupProc	<string>	read/write	Name of Tcl function to create any additional fields/widgets in the GUI. Also can do pre-processing of results if required.
graphTypes	[<string>]	read/write	List of graph types which can be requested for the plot.
modifiers	[<string>]	read/write	List of one-option operator names that can be used to modify the generated expression. If a modifier is chosen, the contents of the rest of the expression will be wrapped in a call to that operator.
updateCustomFieldsProc	<string>	read/write	Name of optional Tcl function for updating Widgets in the Plot Assistant's GUI form after results files have changed.
checkCustomFieldsProc	<string>	read/write	Name of optional Tcl function for determining whether selection can start. Must be provided only if the expressionDef has custom fields.
createExpressionProc	<string>	read/write	Name of Tcl function to generate and return the expression.
resultTypes	<string>	read/write	List of simulation result types that apply to the function definition. For example, {tran ac dc}.

Callbacks

This section covers the following topics:

- [Simulator Callbacks](#)
- [Analysis Callbacks](#)
- [Environment Option Callbacks](#)
- [Save Option Callbacks](#)
- [Data Map Callbacks](#)

Simulator Callbacks

The following table describes the callbacks for the `sa::<simulator>` namespace:

Name	Arguments	Description
<code>createFinalNetlistProc</code>	<code><saTestbench></code>	The callback procedure for creating the final netlist for simulation.
<code>runCommandProc</code>	<code><saTestbench></code>	The callback procedure for getting the command line arguments for running simulation.
<code>runScriptProc</code>	<code><saTestbench></code>	The callback procedure for getting the command line arguments for writing to the file <code>runSimulation</code> in the netlist directory.
<code>createResultsMapProc</code>	<code>{<saAnalysisInfo>}</code> <code><saTestbench></code> <code><saResults></code>	The callback procedure called at the end of the simulation run for creating the results map files.
<code>getLogFilesProc</code>	<code><saTestbench></code>	The callback procedure for getting the log files in addition to the value of the <code>logFile</code> attribute of the <code>saSimulator</code> object.
<code>mapOpPointParamProc</code>	<code><name></code> <code><param></code> <code><netlistDir></code> <code><outputFormat></code>	This callback procedure, which is called only for the result types <code>opBegin</code> and <code>opTran</code> , returns the mapped name of the op parameter. The mapped parameter name will be used in the device label assistant.
<code>simulationDoneProc</code>	<code><saTestbench></code> <code><status></code>	This callback procedure is called after simulation is completed. The possible values for the "status" argument are "success" and "failure."
<code>getGroundNetAliasProc</code>	<code><netName></code> <code><netlistDir></code>	This callback procedure is called to get the alias name for the ground net if the signal is not found in the output files with the name mapped in simulator namespace. The <code><netName></code> is passed in the simulator namespace.
<code>getAnalysesGroups</code>		The callback procedure for defining named groups of analyses for placing on different tabs in the Edit/Create Analyses dialog box. Returns a list of name/analysis-list pairs. Tab groups and their analysis types will appear in the GUI in the order this function returns them in. Empty strings within a list of analysis type names can be used to divide the list between different rows in the radio button.

Analysis Callbacks

The following table describes the callbacks for the `sa::<simulator>::<analysis_type>` namespace:

Name	Arguments	Returns	Description
getAttrs		{<field_name> <type> <label> [<enum>]}	The callback procedure for registering the analysis field definition. This includes analysis field, field type, field label, optional preference name, and optional choices for the field value for each analysis fields.
createDialog	<parent>	No	The callback procedure for creating the widget for each analysis field. The fields should be created inside the container specified in the argument <parent>.
getDisplayFields	<saAnalysis>	{<string>}	The callback procedure for returning the list of analysis field names which will be displayed when the analysis item is expanded in the analyses pane.
synopsis	<saAnalysis>	<string>	The callback procedure for getting the analysis summary line when the analysis item is collapsed in the analyses pane.
getTests	<saAnalysis>	{<string>}	The callback procedure for returning the list of the implemented test procedures for validating the analysis field values for errors.
depends	<saTestbench> <saAnalysis>	<bool>	The callback procedure which returns true if the analysis depends on having different analysis being enabled in the given testbench. For example, noise analysis depends on having ac analysis enabled.
supportsMultiple		<bool>	The callback procedure which returns true if the multiple analyses of same analysis type is supported.
getResultTypes		{<string>}	A callback procedure to define what result types are produced by an analysis. If not provided, result type assumed to be identical to analysis type.

Environment Option Callbacks

The following table describes the callbacks for the `sa::<simulator>::envOptions` namespace:

Name	Arguments	Returns	Description
getAttrs		{<name> <type> <label> [<enum>]}	The callback procedure for registering the additional environment options for the target simulator. This includes option name, option type, label, optional preference name, and optional choices for the option value for each additional option.
createDialog	<parent> <saEnvironmentOptions>	No	The callback procedure for creating the widget for each additional environment option. The widget should be created inside the container specified in the argument <parent>. The default value for the widget is specified in the corresponding attribute in the <saEnvironmentOptions> object.
validate	<giDialog>	<Boolean>	The callback procedure which checks whether option values are in proper range.

Save Option Callbacks

The following table describes the callbacks for the `sa::<simulator>::saveOptions` namespace:

Name	Arguments	Returns	Description
getAttrs		{<name> <type> <label> [<enum>]}	The callback procedure for registering the save options for the target simulator. This includes name, type, label, optional preference name, and optional choices for the save option value for each save option.
createDialog	<parent> <saSaveOptions>	No	The callback procedure for creating the widget for each attribute of the saSaveOptions. The widget should be created inside the container specified in the argument <parent>. The default value for the widget is specified in the corresponding attribute in the <saSaveOptions> object.
validate	<giDialog>	<Boolean>	The callback procedure which checks whether save option values are in proper range.

Data Map Callbacks

The following table describes the callbacks for the `sa::<simulator>::<result_type>` namespace:

This callback procedure returns a signal name to match the name in the simulator namespace.

Name	Arguments	Returns	Description
dataMap	<name> <param> <dataType> <netlistDir> <outputFormat>	<name>	This callback procedure is called before accessing the signal from the simulator output file. The returned signal name is used by the viewer integration for accessing the waveform data from the output file.

Error Checking Procedures

The following table shows the error checking functions in the `sa::check` namespace, which can be used to validate fields in the analysis dialog box:

Function	Arguments	Returns	Description
numeric	<saAnalysis> <attribute_name>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> object is numeric.
notBlank	<saAnalysis> <attribute_name>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> object is not empty.
nonNumeric	<saAnalysis> <attribute_name>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> object is not numeric.
numericGreater	<saAnalysis> <attribute_name> <value>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> is a numeric value greater than <value>.
numericLessThan	<saAnalysis> <attribute_name> <value>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> object is a numeric value less than <value>.
fieldGreater	<saAnalysis> <attribute_name1> <attribute_name2>	<boolean>	Verifies that the value of the <attribute_name1> is greater than value of the <attribute_name2> from <saAnalysis> object.
numericNequal	<saAnalysis> <attribute_name> <value>	<boolean>	Verifies that the value of the <attribute_name1> in <saAnalysis> object is a numeric value not equal to <value>.

Function	Arguments	Returns	Description
numericEqual	<saAnalysis> <attribute_name> <value>	<boolean>	Verifies that the value of the <attribute_name1> in <saAnalysis> object is a numeric value equal to <value>.
numericInRange	<saAnalysis> <attribute_name> <min> <max>	<boolean>	Verifies that the value of the <attribute_name> in <saAnalysis> object is a numeric value within the range <min> and <max>.
netExists	<saAnalysis> <attribute_name>	none	Verifies that the value of the <attribute_name> in <saAnalysis> object is a valid net name.
instExists	<saAnalysis> <attribute_name>		Verifies that the value of the <attribute_name> in <saAnalysis> object is a valid instance name.
sourceExists	<saAnalysis> <attribute_name>		Verifies that the value of the <attribute_name> in <saAnalysis> object is a valid instance name and the property source is set to t in the simInfo of the component corresponds to the instance name.
createWarning	<saAnalysis> <msg>	none	This procedure is called in <code>sa::<simulator>::<analysis_type>::depends</code> for displaying the warning message <msg> at the bottom of the <saAnalysis> form in the Edit/Create Analysis dialog box when the <saAnalysis> depends on having an enabled analysis of a different analysis type.

Results Analyzer Registration Functions

The following table shows the functions in the `sa::plotAssist` namespace, which can be used to register basic expressions for AC, DC, and Transient Analyses:

Function	Arguments	Description
registerAcCurrentExpressions	simulatorName	Adds 'current' to the expressions list for the AC analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal, and Object. Provides modifiers to plot magnitude, phase, dB10, and dB20 signals.
registerAcSignalExpressions	simulatorName	Adds 'signal' to the expressions list for the AC analysis. Provides selections for IStandard with Instance Currents and Standard with Instance Voltages. Provides modifiers to plot magnitude, phase, dB10, and dB20 signals.

Function	Arguments	Description
registerAcVoltageExpressions	simulatorName	Adds 'voltage' to the expressions list for the AC analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal, and Object. Provides modifiers to plot magnitude, phase, dB10, and dB20 signals.
registerDcCurrentExpressions	simulatorName	Adds 'current' to the expressions list for the DC analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal, and Object.
registerDcPowerExpressions	simulatorName	Adds 'power' to the expressions list for the DC analysis. Provides selections for Instance Terminal, 2-terminal Instance, Terminal and Voltage Reference, Power dissipated by an Instance.
registerDcSignalExpressions	simulatorName	Adds 'signal' to the expressions list for the DC analysis. Provides selections for Standard with Instance Currents, Standard with Instance Voltages, and Standard with Instance Power.
registerDcTotalPowerExpressions	simulatorName	Adds 'total power' to the expressions list for the DC analysis. Adds 'power()' to the Expression buffer for evaluating the expression.
registerDcVoltageExpressions	simulatorName	Adds 'voltage' to the expressions list for the DC analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal and Object.
registerTranCurrentExpressions	simulatorName	Adds 'current' to the expressions list for the Transient analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal and Object.
registerTranPowerExpressions	simulatorName	Adds 'power' to the expressions list for the Transient analysis. Provides selections for Instance Terminal, 2-terminal Instance, Terminal and Voltage Reference, Power dissipated by an Instance.
registerTranSignalExpressions	simulatorName	Adds 'signal' to the expressions list for the Transient analysis. Provides selections for Standard with Instance Currents, Standard with Instance Voltages and Standard with Instance Power.
registerTranTotalPowerExpressions	simulatorName	Adds 'total power' to the expressions list for the Transient analysis. Adds 'power()' to the Expression buffer for evaluating the expression.

Function	Arguments	Description
registerTranVoltageExpressions	simulatorName	Adds 'voltage' to the expressions list for the Transient analysis. Provides selections for Instance Terminal, Differential Terminal, Instance 'PLUS' Terminal and Object.

Options XML Schema Definition File

The following XML Schema Definition (XSD) describes the format and structure of the simulator options and analyses definition file:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<!-- definition of option type -->
<xs:simpleType name="optionType">
  <xs:restriction base="xsd:string">
    <xs:enumeration value="int"/>
    <xs:enumeration value="bool"/>
    <xs:enumeration value="float"/>
    <xs:enumeration value="string"/>
    <xs:enumeration value="file"/>
    <xs:enumeration value="dir"/>
  </xs:restriction>
</xs:simpleType>
<!-- definition of limit type -->
<xs:simpleType name="limitType">
  <xs:restriction base="xsd:string">
    <xs:enumeration value="intRange"/>
    <xs:enumeration value="floatRange"/>
    <xs:enumeration value="enum"/>
  </xs:restriction>
</xs:simpleType>
<!-- definition of option definition -->
<xs:complexType name="optionDef">
  <xs:attribute name="name" type="xs:string" use="optional" />
  <xs:attribute name="type" type="optionType" use="optional" />
  <xs:attribute name="optional" type="xs:boolean" default="false" />
  <xs:attribute name="mustEnable" type="xs:boolean" default="false" />
  <xs:attribute name="supportsExpression" type="xs:boolean"
default="false" />
  <xs:element name="defaultValue" type="xs:string" />
  <xs:element name="prompt" type="xs:string" minOccurs="0" />
  <xs:element name="description" type="xs:string" minOccurs="0" />
  <xs:element name="limit">
    <xs:attribute name="type" type="limitType" />
  </xs:element>
  <xs:complexType>
    <xs:element name="minValue" type="xs:string" minOccurs="0"
maxOccurs="1" />
    <xs:element name="maxValue" type="xs:string" minOccurs="0"
maxOccurs="1" />
  </xs:complexType>
</xs:complexType>
</xs:schema>
```

```
<xs:element name="value" type="xs:string"
            minOccurs="0" maxOccurs="unbounded" />
</xs:complexType>
</xs:element>
</xs::complexType>
<xs:element name=comment type="xs:string" minOccurs="0" maxOccurs="1"/>
<xs:element name="optionDef" type="optionDefs" maxOccurs="unbounded" />
<xs:element name="analysisDef" maxOccurs="unbounded">
  <xs:attribute name="name" type="xs:string" />
  <xs:element name="optionDef" type="optionDef" maxOccurs="unbounded" />
</xs:element>
</xs:schema>
```

Using Markup Syntax

The netlist template controls the following netlist parts when creating the final netlist:

- General structure and layout of the overall netlist
- Simulator-specific options and settings which are common to all netlists in *the* integration
- Location of the design variables in the netlist
- Location of the model files sections in the netlist

The following tags are supported:

Tag	Description
[#netname]	Replaces the [#netname] expression with the netlister-assigned node name for netname. The netname may be a hierarchical reference in the design namespace like "inst/net1". Note that since leading hierarchy delimiters are not allowed in certain syntaxes, the leading hierarchy delimiter's exclusion in the target will be based on the xtNameMap's leadingDelimiter attribute value. If the user omits the "/" in the original PrimeWave Design Environment name-space path, the leading delimiter in the target will also be excluded.
[\$instname]	Replaces the [\$instname] expression with the netlister- assigned instance name for instname. The instname may be a hierarchical reference in the design namespace like "inst1/inst2".
[+terminal]	Replaces the [+terminal] with the netlister-assigned node name of the net attached to the specified terminal. The terminal may be a hierarchical reference in the design namespace like "inst1/inst2:plus". Note, if a terminal name is specified without a hierarchical path, or following a hierarchy delimiter, then it is the terminal of a subcircuit as in [+in1] or [+/inst/in2]. There are still some issues of how to map names of terminals on devices.

Tag	Description
[!filename]	Replaces the [!filename] expression with the contents of the filename file after being processed for further name mapping and file substitution. If a relative pathname is used, the system will look for the filename file in the run directory. A full file system pathname can be used to access a file outside the run directory. An error will be generated if the filename file does not exist.
[?filename]	Replaces the [?filename] expression with the contents of the filename file after being processed for further name mapping and file substitution. If a relative pathname is used, the system looks for the filename file in the run directory. To access a file outside the simulation run directory, use a full file system pathname. No error is generated if the file does not exist.
[n!filename]	Identical to [! filename], except that the contents of the new file are not parsed, and netlist tags are not interpreted.
[n?filename]	Identical to [? filename], except that the contents of the new file are not parsed, and netlist tags are not interpreted.